RFC - ML Project Proposal: Retrieval-Augmented Product Question Answering

Author: Manoj Patra

Project Background

This project proposes an AI product question-answering system for an e-commerce setting. The model's objective is to answer customer questions about a product by retrieving the most relevant product information, prior customer questions, and existing answers, then selecting or generating a concise answer that is grounded in the available evidence.

This problem is more challenging than a basic classification or regression project because it reflects a current AI use case: retrieval-augmented generation (RAG) for customer support and shopping assistants. It involves semantic search, answer relevance ranking, confidence scoring, and hallucination control. At the same time, it is still feasible for the course because the first version can avoid training a large language model from scratch. The core ML system can be built with embeddings, candidate retrieval, and a lightweight supervised ranking/classification model.

The primary machine learning problem will be **natural language processing and information retrieval**, specifically **supervised answer relevance ranking**. Given a product question and multiple candidate pieces of product evidence, the model will rank which candidate is most likely to answer the question. As an extension, the system can use a small language model or API-based LLM to generate a final answer only from the top-ranked evidence.

Technical Background

The data source will be **Amazon-PQA** from the **Registry of Open Data on AWS**. The dataset contains Amazon product questions and answers along with public product information. I may also use the related contextual product QA resources from Amazon

Science, which include product-question datasets with candidate evidence, source information, context, relevance labels, and manually written answers. These fields are useful because they allow the project to evaluate retrieval/ranking separately from answer generation.

Data preparation will include loading a manageable subset of the product QA data, cleaning question and answer text, removing duplicates, normalising product identifiers, filtering empty or low-quality records, and creating candidate evidence pairs for each question. I will generate text embeddings for questions and candidate product evidence, then build a retrieval index so the system can return the top candidate answers or evidence snippets for each question.

The first baseline will use keyword search or TF-IDF/BM25 retrieval. A stronger version will use sentence embeddings for semantic retrieval, followed by a supervised relevance model such as logistic regression, Random Forest, XGBoost/LightGBM, or a small transformer cross-encoder if time permits. The main features will include question text, candidate answer/evidence text, product title/category, evidence source, embedding similarity score, lexical overlap, question type, answer length, and historical relevance labels if available.

I will evaluate the model using held-out questions and relevance labels. Retrieval and ranking metrics will include **Recall@K**, **Precision@K**, **Mean Reciprocal Rank (MRR)**, and **NDCG@K**. If answer generation is included, I will evaluate answer quality using semantic similarity to reference answers, groundedness checks against the retrieved evidence, and a small human review rubric for correctness, usefulness, and unsupported claims.

For monitoring, I would track input data quality, schema changes, missing fields, duplicate questions, changes in product categories, question-topic drift, embedding similarity distributions, retrieval confidence, no-answer rate, user feedback, latency, and the rate of answers flagged as unsupported. A retraining or re-indexing trigger could be defined when retrieval metrics drop, user feedback declines, or the distribution of incoming questions shifts significantly from the training data.

Goals vs Non-Goals

Goals

- Build a reproducible pipeline for loading product QA data, cleaning text, creating embeddings, retrieving candidates, and evaluating ranked results.
- Compare a simple keyword retrieval baseline with a semantic retrieval/ranking approach.

- Train or configure a model that can identify which candidate evidence best answers a product question.
- Define measurable offline evaluation metrics for retrieval quality and answer quality.
- Include a practical monitoring plan for drift, confidence, unsupported answers, and user feedback.

Non-Goals

- This project will not train a large language model from scratch.
- This project will not build a full production chatbot or shopping website.
- This project will not support every product category in the first version; a selected category or subset is acceptable.
- This project will not rely on unsupported generated answers without retrieved evidence.
- This project will not optimise for real-time serving at enterprise scale in the initial prototype.

Sources

- Amazon-PQA - Registry of Open Data on AWS
- Learning to Rank and Filter - Community Question Answering - Registry of Open Data on AWS
- Amazon Science Contextual Product Question Answering repository

General System Services for project deliverables:

1. **S3 Bucket**
2. **Sagemaker notebook**
3. **Sagemaker feature store**
4. **Container registry**
5. **Cloudwatch**
6. **Autoscaling***
7. **Sagemaker endpoint**
8. **Sagemaker Groundtruth**
9. **EC2 Instance***
10. **Amplify**
11. **Flask Server**

Other considerations:

1. Infrastructure monitoring dashboard
2. Model quality monitoring baseline
3. Model quality monitoring setup
4. Model quality monitoring alarm
5. CI/CD DAG Overview
6. Web app
7. Model registry